Apuntes – MVVM y Repository

Desarrollo de Aplicaciones I · UADE · Guía de Ejercicios: MVVM, Repository y archivos (v1.0)

Repositorio: `MVVMlab` · una rama por ejercicio (`ej01-…` a `ej15-…`) · Felipe Forte · Kotlin + Jetpack Compose

---

Cómo leer estos apuntes. La **Parte 1** resume la teoría (presentación *MVVM con Kotlin y Jetpack Compose*) y los temas más importantes. La **Parte 2** recorre los ejercicios uno por uno: consigna, "En pocas palabras", **"Importante a considerar"**, la explicación con el código real del proyecto, las pruebas en el emulador y las preguntas de la guía. Al final hay **anexos** (organización, cómo ejecutar y probar, preguntas transversales, registro de uso de IA) y el **glosario**.

> **Uso de IA (secciones 1.1 y 19 de la guía)**
> La IA se permite para consultar errores, diagramas, pruebas o fragmentos concretos, sin aceptar propuestas sin revisar. Si se usa para resolver un bloqueo, la guía pide conservar: **problema, fragmento enviado, explicación recibida, cambio realizado y prueba** con que se verificó (plantilla en el Anexo D).

Contenido

1. **Parte 1 · Introducción teórica**
2. **Parte 2 · Ejercicios**
   - Ejercicio 1 — ¿Quién está haciendo demasiado?
3. **Anexos**
   - Anexo A · Organización del proyecto en carpetas
   - Anexo B · Cómo ejecutar y probar
   - Anexo C · Preguntas transversales (sección 18 de la guía)
   - Anexo D · Registro breve de uso de IA (sección 19 de la guía)
4. **Glosario**

> **Versión de esta rama**
> Este PDF llega hasta el **Ejercicio 1**. La versión completa está en la rama `main`.

Parte 1 · Introducción teórica

Fuente: presentación de cátedra *MVVM con Kotlin y Jetpack Compose — arquitectura de presentación, estado y programación reactiva*. El punto de partida es un síntoma: una pantalla que obtiene datos, los transforma, valida, guarda y además dibuja. Cada interacción agrega variables y condiciones, y una rotación o un error vuelven difícil saber qué estaba viendo el usuario. **MVVM no nace para agregar clases: busca delimitar responsabilidades y hacer explícito quién es el dueño del estado.**

1. Tres responsabilidades

```
View (Compose) ──intención──► ViewModel ──operación──► Model / datos (Repository)
View (Compose) ◄──estado─── ViewModel ◄────datos──── Model / datos
```

Pieza	Le corresponde	Evitar
View Compose	Dibujar el estado, emitir clics, estado visual local	Consultar la base o archivos desde el composable
ViewModel	Recibir la intención, coordinar, validar, exponer `UiState`	Conocer la Activity o modificar widgets
Repository	Proveer datos (observables) y operaciones de escritura	Saber cómo se dibuja la pantalla

El Repository pertenece a la **capa de datos**; MVVM organiza sobre todo la presentación. Una capa de dominio aparte es opcional.

2. UDF y Observer

- **Unidirectional Data Flow**: los eventos **suben** de la UI al dueño del estado; el estado nuevo **baja** para ser dibujado. No hay varias piezas modificando la misma lista por separado.
- **Observer**: la UI observa un productor (un `StateFlow`) y reacciona cuando cambia; no pregunta todo el tiempo.
- **Programación reactiva**: *UI = f(estado)*. Se modelan cambios observables, no pasos para refrescar controles.

3. Estado y evento

	Estado	Evento
Idea	Qué es verdad ahora	Algo que ocurrió
Ejemplo	“Hay dos notas sin guardar”	“El usuario tocó Guardar”
Quién	Compose lo representa	El ViewModel lo procesa y produce estado nuevo

¿Dónde vive el estado? `remember` (recomposiciones) → `rememberSaveable` (recreación) → **ViewModel** (rotación; *no* sobrevive a la muerte del proceso) → **Repository + persistencia** (más allá de la pantalla). **State hoisting**: el valor y un callback se elevan al ancestro dueño.

4. Kotlin para modelar estado

- `data class` + `copy(...)`: estados inmutables; se publica una versión nueva en vez de mutar una colección compartida.
- `sealed interface`: alternativas excluyentes (*Cargando*, *Disponible*, *Fallo*) que el `when` obliga a cubrir; evita combinaciones imposibles de banderas sueltas. No reemplaza a los campos independientes (búsqueda, filtro).
- Lambdas/callbacks para avisar interacciones; tipos anulables para la ausencia de valor.

5. Corrutinas y Flow

- **Corrutina**: trabajo que puede suspenderse sin bloquear la UI. `suspend fun` marca que puede esperar; `viewModelScope.launch` la inicia y se cancela cuando el ViewModel se destruye. Para disco: `withContext(Dispatchers.IO)`.
- **Flow**: secuencia de valores en el tiempo. **Frío** (arranca al recolectar) vs **caliente** (existe aunque nadie lo mire).
- **StateFlow**: caliente, siempre con valor actual; ideal para el `UiState`. El ViewModel guarda un `MutableStateFlow` privado y expone `asStateFlow()`; actualiza con `_uiState.update { it.copy(...) }`.
- **SharedFlow**: difusión sin valor inicial; una señal puede perderse si nadie observa (no usarla para resultados importantes).
- `collectAsStateWithLifecycle()`: Compose observa el flujo solo mientras la pantalla está activa.
- `stateIn(scope, WhileSubscribed(5_000), inicial)`: convierte el `Flow` del Repository en estado de pantalla.
- Operadores: `map`, `filter`, `combine`, `debounce`, `flatMapLatest`, `catch`; usarlos solo si hay un comportamiento que lo pida.

6. Repository y dependencias

```
interface NotaRepository {                    // contrato chico y estable
    fun observarNotas(): Flow<List<Nota>>
    suspend fun guardar(nota: Nota)
}
class MiViewModel(private val repo: NotaRepository) : ViewModel()   // recibe el contrato, no la
implementación
```

La implementación puede ser memoria, archivo, Room o una falsa para tests: la pantalla no cambia. Quién crea el Repository y se lo da al ViewModel es una decisión aparte (**inyección de dependencias**: un contenedor en el `Application` y una *factory* del ViewModel). Probar el ViewModel = reemplazar el Repository por un *fake* y verificar el estado publicado, sin emulador.

Temas más importantes a considerar

1. **Cada pieza una responsabilidad**: la View dibuja y avisa; el ViewModel decide y expone estado; el Repository guarda y obtiene datos.
2. **UDF**: eventos hacia arriba, estado hacia abajo; una sola fuente de verdad por dato.
3. **Estado inmutable** en un `StateFlow` privado/público con `copy`; la pantalla solo lo observa.
4. **ViewModel ≠ persistencia**: sobrevive a la rotación, no a la muerte del proceso; los datos importantes van al Repository.
5. **Estados de carga con `sealed`**: Cargando / Listo / Error, sin banderas contradictorias.
6. **Disco fuera del hilo principal** (`Dispatchers.IO`) y errores comunicados al usuario, no solo en Logcat.
7. **Depender de contratos** (interfaces) y crear las dependencias en un solo lugar.
8. **Probar el ViewModel sin Android** con repositorios falsos.

Parte 2 · Ejercicios

Cada ejercicio vive en su propia rama y contiene solo su código, organizado por capas (ver Anexo A).

Ejercicio 1 — ¿Quién está haciendo demasiado?

Consigna: tomar una pantalla donde todo está mezclado, marcar dónde se dibuja la interfaz, dónde se validan los datos, dónde se modifica la lista y dónde se guarda, y ubicar cada responsabilidad en *View*, *ViewModel* o *Model/Repository*. Todavía **no** se cambia la implementación.

En pocas palabras

Se analiza una pantalla que hace **todo junta** (dibujar, validar, guardar) y se decide a qué parte de MVVM pertenece cada línea. No se escribe código nuevo: es un ejercicio de *mirar y clasificar*. Es la base de todo lo que sigue.

Importante a considerar

- **"Declarar una variable" no dice nada de la capa.** Hay variables en la View, en el ViewModel y en el Repository. La pregunta útil es: *¿este dato tiene que sobrevivir si cierro la app?*
- **El ViewModel decide QUÉ mostrar; la View decide CÓMO.** "Hay un error" es del ViewModel; "el error va en rojo" es de la View.
- **Una regla de negocio** es algo que valdría igual si la app fuera web ("no se aceptan tareas vacías"). Esas van fuera de la pantalla.
- **Evento ≠ estado.** El click en "Agregar" es un evento (pasó); la lista de tareas es estado (está). Los eventos suben, el estado baja.
- **Ver un `Context` dentro de una pantalla es una pista** de que algo (por ejemplo, guardar archivos) está en el lugar equivocado.

Conceptos base

MVVM = separar en 3 responsabilidades

- `View (Compose)` **dibuja** el estado y **avisa** lo que hace el usuario (clicks, texto). No decide nada.
- `ViewModel` **recibe** la acción del usuario, **decide** (valida, coordina) y **expone** el estado de pantalla.
- `Model / Repository` **obtiene y guarda** los datos (memoria, archivo, Room, API). No sabe nada de pantallas.

UI = f(estado): la pantalla es una función del estado.

¿Cómo decido dónde va cada cosa?

El criterio **no** es "¿declara una variable?" — hay variables en las tres capas. Las preguntas útiles son:

1. **¿Esto dibuja algo?** → View. *(Test: si cambio Compose por una web, ¿tengo que reescribir esta línea? Sí → View.)*
2. **¿Es una regla que valdría igual en cualquier interfaz?** → ViewModel.
3. **¿Este dato tiene que sobrevivir si cierro la app?** Sí → dato real → Repository. No, solo importa mientras uso la pantalla → estado de pantalla → ViewModel.

Ejemplo: en WhatsApp, el mensaje que estás escribiendo (borrador) es de la pantalla; los mensajes ya enviados se guardan.

> **El ViewModel decide QUÉ mostrar. La View decide CÓMO mostrarlo.**
> “Hay un error: tarea vacía” → ViewModel. “El error va en rojo, debajo del botón” → View.

Código analizado (pantalla “mal hecha a propósito”)

```
@Composable
fun PantallaTareas(context: Context, modifier: Modifier = Modifier) {
    var texto by remember { mutableStateOf("") }                    // (A)
    var tareas by remember { mutableStateOf(listOf<String>()) }     // (B)
    var error by remember { mutableStateOf<String?>(null) }         // (C)

    Column(modifier.padding(16.dp)) {                                // (D)
        Text("Mis tareas (${tareas.size})")
        OutlinedTextField(value = texto, onValueChange = { texto = it })

        Button(onClick = {
            if (texto.isBlank()) {                                   // (E)
                error = "La tarea no puede estar vacía"
            } else if (tareas.contains(texto)) {                     // (F)
                error = "Esa tarea ya existe"
            } else {
                tareas = tareas + texto                              // (G)
                File(context.filesDir, "tareas.txt")                 // (H)
                    .writeText(tareas.joinToString("\n"))
                texto = ""                                           // (I)
                error = null
            }
        }) { Text("Agregar") }

        error?.let { Text(it, color = Color.Red) }                   // (J)
        LazyColumn { items(tareas) { Text(it) } }                    // (K)
    }
}
```

Resultado: tabla de responsabilidades (entregable)

Letra	Código	Qué hace	Dónde va
(A)	`var texto`	Borrador que escribe el usuario; no sobrevive al cierre	ViewModel
(B)	`var tareas`	Lista real de tareas; tiene que sobrevivir al cierre	Repository
(C)	`var error`	Mensaje de error actual (dato del estado de pantalla)	ViewModel
(D)	`Column`, `Text`, `OutlinedTextField`	Dibuja la pantalla	View
(E)	`texto.isBlank()`	Regla: no se agregan tareas vacías	ViewModel
(F)	`tareas.contains(texto)`	Regla: no se agregan tareas repetidas	ViewModel
(G)	`tareas = tareas + texto`	Agrega la tarea a la lista (el dueño del dato la modifica)	Repository
(H)	`File(...).writeText(...)`	Guarda las tareas en un archivo	Repository
(I)	`texto = ""`	Limpia el campo solo si la tarea se agregó bien (decisión sobre un dato del ViewModel)	ViewModel
(J)	`error?.let { Text(it, Color.Red) }`	Muestra el error en rojo (el error es del VM; dibujarlo es de la View)	View
(K)	`LazyColumn`	Muestra la lista en pantalla	View

Errores frecuentes (los que cometí y corregí)

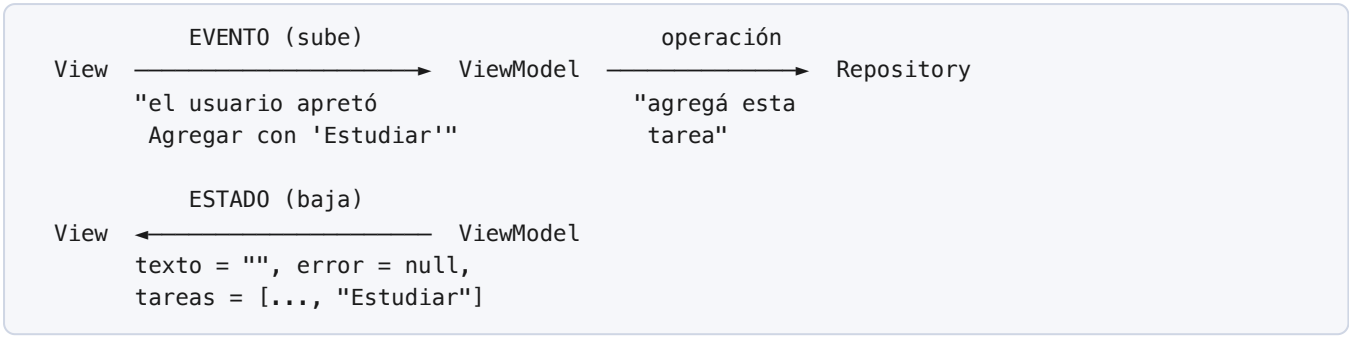
- Pensar que (A), (B) y (C) van al Repository "porque declaran variables". Solo (B) es un dato que debe persistir.
- Poner (I) en la View: `texto` es del ViewModel, y quien es dueño de un dato es quien lo modifica.
- Poner (J) en el ViewModel: el *error* es del ViewModel, pero *mostrarlo* con `Text` y `Color.Red` es de la View.

La cadena completa de "Agregar"

1. View el usuario aprieta *Agregar* → avisa al ViewModel.
2. ViewModel aplica las reglas (E) y (F). Si pasan → le pide al Repository que agregue; limpia el campo (I).
3. Repository agrega la tarea (G) y la guarda en el archivo (H).

Repository = **dueño del dato** · ViewModel = **intermediario** (lo pide y lo prepara) · View = **solo lo muestra**. La lista "verdadera" vive en un solo lugar: **fuente de verdad única**.

Desafío: las dos flechas (UDF – flujo unidireccional de datos)



> **Evento vs. estado**
>
> - **Evento** = algo que *pasó* (un click, una tecla). Es puntual.
> - **Estado** = cómo *está* la pantalla ahora. Se puede consultar en cualquier momento.
> - **UDF:** los eventos suben y el estado baja. La View nunca modifica el estado por su cuenta: solo avisa y redibuja.

Preguntas de la guía

1. ¿Qué trabajo tendría que seguir funcionando aunque mañana cambiemos Compose por otra interfaz?
Las **reglas** (E, F: no vacía, no repetida) y el **manejo de datos** (G, H: agregar y guardar). Es decir, todo lo que va en ViewModel y Repository.

2. ¿Qué datos existen sólo mientras se está escribiendo un formulario?
`texto` (el borrador) y `error` (el mensaje actual). Son estado de pantalla y no se guardan en el archivo.

> **Señal de alarma: `Context` en un composable**
>
> `PantallaTareas` recibe un `Context` solo para poder escribir el archivo (H). La guía advierte no aceptar soluciones que "llevan Context a todos los composables" o "abren archivos desde un botón". Es una pista de que el acceso a archivos está en el lugar equivocado: debería estar en el Repository.

Kotlin / Compose que apareció en el camino

Elemento	Qué es
`Unresolved reference: X`	Kotlin no sabe qué es X: falta el `import`, está mal escrito o la función/clase no existe.
`var x by remember { mutableStateOf(...) }`	Estado observable que sobrevive a la recomposición. El `by` necesita los imports `getValue` / `setValue`.
`LocalContext.current`	Forma de obtener el `Context` de Android dentro de un composable.
`modifier: Modifier = Modifier`	Convención: permite que quien llama le pase padding, tamaño, etc. (por ejemplo el `innerPadding` del Scaffold).
`@Preview` / `@Composable`	Anotaciones: "etiquetas" que siempre se aplican a una función. Solas, sin función debajo, no compilan.

Anexo A · Organización del proyecto en carpetas

El código está organizado por **capas**, con la misma estructura que el repositorio de referencia *PokedexAcademy-MVVM-Clean*. Cada carpeta es un *paquete* de Kotlin (`com.example.mvvmlab.presentation.nota`, etc.). En cada rama aparecen solo las carpetas que ese ejercicio usa.

```
app/src/main/java/com/example/mvvmlab/
 ├── MainActivity.kt          # punto de entrada: aloja la UI
 ├── MvvmLabApp.kt            # Application propia (desde el Ej 11)
 ├── core/
 │    ├── model/              # tipos comunes a varias pantallas (EstadoCarga)
 │    ├── navigation/         # rutas de navegación (desde el Ej 12)
 │    ├── theme/              # tema de Compose (Color, Theme, Type)
 │    └── util/               # utilidades (Fechas)
 ├── data/                    # CÓMO se guardan los datos
 │    ├── local/              # Room: Entity, DAO, Database (Ej 14–15)
 │    ├── mapper/             # Entity ⇄ modelo de dominio
 │    └── repository/         # implementaciones de los contratos (memoria, archivos, Room)
 ├── domain/                  # QUÉ es la app: Kotlin puro, sin Android
 │    ├── model/              # Nota, Registro, ResultadoLectura
 │    ├── repository/         # contratos (interfaces) de los Repository
 │    └── validacion/         # reglas de negocio (ValidadorNota, ValidadorRegistro)
 ├── presentation/            # MVVM: una carpeta por pantalla (UiState + ViewModel + Pantalla)
 └── di/                      # quién crea las dependencias (ContenedorApp, Ej 11)

app/src/test/java/com/example/mvvmlab/
 ├── domain/validacion/       # tests de reglas
 ├── fake/                    # Repository falsos para tests
 ├── presentation/            # tests de ViewModels
 └── util/                    # MainDispatcherRule
```

Capa	Regla
`domain/`	No importa nada de Android, Compose, Room ni archivos. Define los datos (`model`), los contratos (`repository`) y las reglas (`validacion`).
`data/`	Implementa los contratos de `domain`. Es la única capa que conoce archivos, SQL o Room.
`presentation/`	ViewModels y pantallas. Solo habla con los contratos de `domain`, nunca con `data` directamente.
`di/`	Conecta todo: elige qué implementación de `data` recibe cada ViewModel.

> **¿Y los *use cases* del repositorio de referencia?**
>
> El repositorio de referencia tiene además `domain/usecase/`. Acá no se agregaron: la guía pide no crear capas “porque sí”, y en estos ejercicios cada use case solo llamaría a un método del Repository. La teoría (diapositiva 4) aclara que la capa de dominio es opcional.

Anexo B · Cómo ejecutar y probar

```
git clone https://github.com/feliforte14/MVVMlab.git
cd MVVMlab
git checkout ej15-cuaderno-de-campo        # o la rama del ejercicio
```

1. Abrir en Android Studio, Gradle sync y **Run ▶**.
2. Tests unitarios (ViewModels, validador, repositorios falsos): `./gradlew :app:testDebugUnitTest`.
3. Para probar la persistencia: cerrar la app desde Recientes y volver a abrirla; para la muerte del proceso: `adb shell am kill com.example.mvvmlab` con la app en segundo plano.
4. Ver archivos y base: *Device Explorer* (`/data/data/com.example.mvvmlab/files`) y *App Inspection → Database Inspector*.

Anexo C · Preguntas transversales (sección 18 de la guía)

Pregunta	Respuesta
1. ¿La pantalla presenta estado o también decide cómo guardarlo?	Desde el Ej 3 solo presenta: el ViewModel decide y desde el Ej 6 el Repository guarda.
2. ¿Qué clase conoce Context? ¿Lo necesita realmente?	Solo el contenedor de dependencias (en el `Application`, Ej 11) para ubicar `filesDir` y crear la base; los ViewModels no lo reciben. La View lo usa únicamente para lanzar Intents (Ej 15).
3. ¿Qué sobrevive a recomposición, rotación, muerte del proceso y reinicio?	Recomposición: `remember`. Rotación: ViewModel y `rememberSaveable`. Muerte del proceso y reinicio: solo lo escrito en disco (archivos, Room) — Ej 3, 8 y 14.
4. ¿Se accede a disco fuera del hilo principal?	Sí: los repositorios de archivo y Room trabajan en `Dispatchers.IO` / funciones `suspend` (Ej 7 en adelante).
5. ¿Un error se comunica al usuario o solo se imprime en Logcat?	Se modela en el estado (tipos sellados de carga y guardado) y se muestra en pantalla (Ej 9).
6. ¿El ViewModel se puede probar reemplazando el Repository?	Sí: recibe el contrato por constructor; los tests usan fakes y una regla para el dispatcher principal (Ej 13).
7. ¿Hay una sola fuente de verdad para cada dato?	Sí: la lista vive en el Repository y las pantallas la observan; en el integrador, la tabla de Room (Ej 12 y 15).
8. ¿Qué cambio de implementación debería ser invisible para Compose?	Pasar de memoria a archivo o a Room: cambia el Repository y el contenedor, no las pantallas (Ej 6, 10 y 14).

Anexo D · Registro breve de uso de IA (sección 19 de la guía)

La guía pide conservar, para cada bloqueo resuelto con IA: **problema, fragmento enviado, explicación recibida, cambio realizado y prueba concreta** con que se verificó. “No alcanza con anotar ‘la IA lo arregló’”.

Problema	Fragmento enviado	Explicación recibida	Cambio realizado	Prueba
Ejemplo: al rotar se perdía la lista	La pantalla con `remember { mutableStateListOf() }`	La Activity se recrea; el estado tiene que vivir en un ViewModel	Lista movida al ViewModel como `StateFlow`	Rotar el emulador: la lista sigue (Ej 3)

Buen pedido (sección 1.1): “Esta pantalla tiene estas tres variables y estas dos operaciones. Ayudame a decidir cuáles pertenecen a la UI y cuáles al ViewModel. No escribas la app entera.”

Glosario

Término	Qué significa, en simple
Activity	Una pantalla de Android. Android la crea y la destruye (por ejemplo, al rotar el teléfono). Nuestra app tiene una sola: `MainActivity`.
Composable	Función marcada con `@Composable` que *describe* un pedazo de pantalla. No devuelve nada: "emite" interfaz.
Recomposición	Cuando un dato que la pantalla usa cambia, Compose vuelve a ejecutar las funciones composables afectadas para redibujar.
Estado	Los datos que determinan cómo se ve la pantalla *ahora* (qué hay escrito, si hay un error, si está cargando).
Evento	Algo que pasó: un click, una tecla. Produce un cambio de estado.
MVVM	Forma de ordenar el código en tres partes: **View** (dibuja), **ViewModel** (decide y guarda el estado de pantalla) y **Model** (los datos y dónde se guardan).
ViewModel	Clase que guarda el estado de una pantalla y su lógica. Sobrevive a la rotación; no sobrevive a que Android cierre la app.
Repository	Clase que sabe *dónde* están los datos (memoria, archivo, base de datos) y ofrece operaciones como guardar o eliminar.
Interface (contrato)	Lista de funciones que una clase promete tener, sin decir cómo las hace. Permite cambiar la implementación sin tocar a quien la usa.
data class	Clase de Kotlin pensada para guardar datos. Trae gratis `copy`, `equals` y `toString`.
val / var	`val` no se puede reasignar; `var` sí.
null y `?`	`null` = "no hay valor". Un tipo con `?` (`String?`) puede ser null; Kotlin obliga a contemplarlo.
Lambda	Función sin nombre que se pasa como dato: `{ texto -> println(texto) }`. Se usa para avisar eventos (`onClick = { ... }`).
Corrutina	Tarea que puede esperar (leer un archivo, una demora) *sin congelar* la pantalla. Se lanza con `launch`.
suspend	Marca una función que puede "pausarse" mientras espera. Solo se llama desde una corrutina u otra función suspend.
Flow / StateFlow	Un "canal" de valores que cambian con el tiempo. Quien lo observa recibe cada valor nuevo. `StateFlow` siempre tiene un valor actual.
Fábrica (Factory)	Objeto que le explica a Android cómo construir un ViewModel que necesita parámetros.
filesDir	Carpeta privada de la app en el teléfono. Ahí se guardan archivos sin pedir permisos.

Room	Biblioteca de Android para usar una base de datos SQLite con tablas, consultas y Kotlin.